

Rapport Personnel – Itération 1

Projet Long IDE7 – Partie Debug & Exécution

ELFANAOU AHMED

Mai 2026

1 Introduction

Notre équipe travaille sur le projet long IDE7, un environnement de développement destiné à faciliter l'écriture, la compilation, l'exécution et le débogage de programmes Java.

Je fais partie de la partie **Debug & Exécution**. Mon travail a consisté à développer et tester plusieurs composants du module Debug afin de permettre la compilation, l'exécution, la détection d'erreurs, le suivi mémoire et les premiers mécanismes de breakpoints.

2 Travail réalisé

J'ai commencé par récupérer la branche `debug` du projet et vérifier que les fichiers du module étaient bien présents dans l'arborescence du projet. J'ai ensuite exécuté les premiers tests du module à l'aide de la classe `MonTest`.

Une première erreur venait du chemin utilisé pour compiler `Helloworld.java`. Le programme cherchait l'ancien chemin :

```
src/debug/Helloworld.java
```

alors que le fichier réel se trouvait dans :

```
src/main/java/fr/inptoulouse/sn/ide7/debug/Helloworld.java
```

J'ai corrigé ce chemin dans `MonTest.java`, ainsi que le nom complet de la classe Java utilisée pour l'exécution. Après cette correction, les tests de compilation, d'exécution et de `compileAndRun` ont fonctionné correctement.

J'ai ensuite ajouté un système de parsing des erreurs de compilation. Pour cela, j'ai créé les classes :

- `CompilationError`
- `CompilationErrorParser`

Ces classes permettent de transformer une erreur brute produite par `javac` en un message plus lisible. Par exemple, une erreur comme :

```
test_programs/MainError.java:7: error: ';' expected
```

est transformée en :

Ligne 7 : ';' expected

J'ai aussi amélioré la partie mémoire avec la création de `MemorySnapshotManager`. Ce gestionnaire permet de conserver plusieurs états mémoire successifs et de les afficher sous forme d'historique :

Etape 1 : x = 5

Etape 2 : y = 10

Etape 3 : somme = x + y = 15

Ensuite, j'ai créé `DebugController`, une classe servant de passerelle entre le module Debug et la future interface IDE7. Cette classe permet d'appeler simplement le pipeline de compilation et d'exécution depuis une autre partie de l'application.

J'ai également commencé la gestion des breakpoints avec `BreakpointManager`. Cette classe permet d'ajouter, supprimer, afficher et vérifier des breakpoints par numéro de ligne.

Enfin, j'ai développé une première version de breakpoints réels avec l'API JDI de Java. Pour cela, j'ai créé `BreakpointDebugger` et `DebugTarget`. Le test permet de placer un breakpoint sur une vraie ligne de code, de suspendre l'exécution du programme, puis de la reprendre après une action utilisateur.

Le résultat obtenu est :

Breakpoint placé à la ligne 9

Breakpoint atteint à la ligne 9

Programme Java suspendu sur le breakpoint.

Appuie sur Entrée pour reprendre l'exécution...

Reprise de l'exécution.

Cela montre que le programme Java est réellement suspendu sur le breakpoint.

3 Avancement

Les éléments suivants ont été développés et testés :

- Correction du chemin de compilation dans `MonTest`.
- Validation de `CompilerService`.
- Validation de `ExecutionService`.
- Validation du pipeline `compileAndRun`.
- Création de `CompilationError`.
- Création de `CompilationErrorParser`.
- Parsing des erreurs de compilation.
- Création de `MemorySnapshotManager`.
- Affichage d'un historique mémoire.
- Création de `DebugController`.
- Création de `BreakpointManager`.
- Création de `BreakpointDebugger`.
- Test d'un vrai breakpoint avec suspension de l'exécution.
- Tests des exceptions runtime ajoutées par un autre membre.
- Tests du timeout et de l'arrêt d'exécution.

4 Difficultés rencontrées

La première difficulté a été liée aux chemins des fichiers Java. Le projet avait été réorganisé, mais certains tests utilisaient encore d'anciens chemins. Cela empêchait la compilation de fonctionner correctement.

Une autre difficulté a été la gestion des branches Git. Le module Debug et l'interface IDE7 étaient sur des branches différentes, ce qui a nécessité de bien comprendre la différence entre les branches `debug` et `ui`.

J'ai aussi rencontré des conflits Git dans `MonTest.java` après un `git pull`. Il a fallu nettoyer les marqueurs de conflit et conserver les tests utiles.

La partie la plus complexe a été l'utilisation de JDI pour les breakpoints réels. Contrairement à un simple stockage de numéros de ligne, JDI permet de contrôler réellement l'exécution de la JVM. Il a fallu créer une classe cible spécifique, `DebugTarget`, et choisir une vraie ligne exécutable pour que le breakpoint soit atteint.

5 Tests réalisés

Les tests réalisés dans `MonTest` vérifient les fonctionnalités suivantes :

- Compilation de `HelloWorld.java`.
- Détection et parsing d'une erreur de compilation.
- Exécution d'un programme Java compilé.
- Compilation puis exécution automatique.
- Affichage des états mémoire.
- Utilisation de `DebugController`.
- Gestion simple des breakpoints.
- Breakpoint réel avec JDI.
- Parsing des exceptions runtime.
- Vérification de l'arrêt d'exécution et du timeout.

Tous les tests principaux passent correctement.

6 Perspectives

Pour la suite, il faudra intégrer ce module dans l'interface graphique IDE7. Le module Debug est maintenant prêt côté backend, mais il doit encore être connecté aux boutons de l'interface, à l'éditeur de texte et à la console graphique.

Les prochaines améliorations possibles sont :

- intégrer les boutons Compiler, Exécuter et Arrêter dans l'interface ;
- afficher les erreurs directement dans l'éditeur ;
- ajouter un bouton Continuer pour reprendre après un breakpoint ;
- afficher les variables en temps réel ;
- enrichir l'affichage mémoire ;
- stabiliser davantage l'intégration JDI.